

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	B.Jahnavi	Reg. No.:	192325043
Section / Batch:	2023	Date:	

Code of Conduct

I B.Jahnavi / 192325043 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: jahnavi

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.
2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.
3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.
4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.
5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. CFG Trees vs. Dependency Parsing

How the Two Approaches Differ

Context-Free Grammar (CFG) trees represent sentence structure as nested hierarchical constituents (phrases such as NP, VP, PP), built from grammar rewrite rules. Each internal node is a phrase category, and the leaves are the words themselves — the structure emphasizes how words group into larger syntactic units. Dependency parsing, in contrast, represents structure as a set of direct binary relations (edges) between a head word and its dependents (e.g., verb → subject, verb → object, noun → modifier), with no intermediate phrase nodes at all.

Comparison Table

Aspect	CFG (Constituency) Trees	Dependency Parsing
Basic unit	Phrases/constituents (NP, VP)	Direct head–dependent word links
Relationships shown	Implicit, inferred from tree shape	Explicit grammatical relations (subject, object, modifier)
Word order sensitivity	Sensitive to fixed phrase order	More robust to free/flexible word order
Best suited for	Phrase-structure grammar checking	Extracting who-did-what-to-whom relations

Justification

For capturing relationships between words — which is precisely what grammar checking requires — dependency parsing is more effective. It directly encodes which word governs which (e.g., verb→subject agreement, verb→object relations) without requiring the reader to infer relationships from phrase boundaries, and it remains robust for languages or sentences with flexible word order. CFG constituency trees remain valuable for understanding overall phrase structure but are less direct for relation-level grammar analysis.

Q2. Top-Down Parsing vs. Earley Parsing (Real-Time, Incomplete Input)

How the Two Techniques Differ

Top-down parsing starts from the grammar's start symbol and predicts a derivation by expanding rules until it matches the input, retrying (backtracking) when an expansion fails. It requires a complete derivation and cannot naturally handle left-recursive rules. Earley parsing, by contrast, processes input incrementally, left to right, and maintains a dynamic-programming chart of all partial parse states consistent with the input seen so far — it does not commit early to one derivation path.

Comparison Table

Aspect	Top-Down Parsing	Earley Parsing
Strategy	Predictive, top-down expansion with backtracking	Incremental, chart-based dynamic programming
Left recursion	Fails / needs grammar rewriting	Handled natively
Worst-case complexity	Exponential	Polynomial, $O(n^3)$
Partial/incomplete input	Poor — expects full derivation	Handles naturally via incremental states

Which Is More Suitable for Dynamic Input Conditions

Earley parsing is more suitable for real-time applications with incomplete or ambiguous sentences. Its incremental chart naturally supports partial parses at any point in the input stream, and its polynomial-time guarantee avoids the unpredictable backtracking costs of top-down parsing — both essential properties for responsive, real-time language understanding.

Q3. CFG vs. PCFG vs. Neural Parsing — "She saw the man with a telescope"

How the Approaches Differ in Handling Ambiguity

- CFG: generates all syntactically valid parses — here, both "with a telescope" modifying "saw" (she used the telescope) and modifying "the man" (the man had a telescope) — with no mechanism to prefer one over the other.
- PCFG: assigns learned probabilities to grammar rules based on corpus frequency, allowing the parser to statistically rank the two readings and select the more probable one — but this ranking is based purely on rule frequency, not real-world/contextual plausibility.
- Neural Parsing: uses contextual embeddings and learned representations that capture semantic and discourse cues beyond simple rule frequency, enabling it to weigh world knowledge and sentence context (e.g., prior mentions of a telescope) to resolve ambiguity more robustly, including for novel or context-dependent cases PCFG has never seen.

Most Effective Method for Real-World Applications

Neural parsing is the most effective for real-world disambiguation, since it captures rich contextual and semantic signals that plain frequency-based statistics (PCFG) cannot, and generalizes better to sentences or domains not well represented in the training grammar. PCFG remains a useful, lighter-weight, more interpretable fallback where computational resources or training data for neural models are limited, while plain CFG is the weakest choice since it cannot resolve ambiguity at all.

Q4. Feature Structures vs. Sub-Categorization Frames

How the Two Approaches Differ

Feature structures are attribute–value matrices (e.g., number: singular, person: 3rd, tense: present) attached to words or constituents, checked for consistency via unification. Their role is to enforce agreement — for example, confirming that a third-person-singular subject correctly matches a correspondingly-inflected verb ("he runs" vs. "he run"). Sub-categorization frames, by contrast, specify the number and type of arguments (complements) a verb requires — for example, "give" requires both a direct and an indirect object, while "sleep" requires none. Their role is to enforce correct argument/complement structure rather than agreement.

Comparison Table

Aspect	Feature Structures	Sub-Categorization Frames
Purpose	Enforce agreement (number, person, tense)	Enforce required verb arguments/complements
Mechanism	Unification of attribute-value pairs	Verb lexicon entries listing required argument types
Example error caught	"He run" (agreement mismatch)	"He gave" with no objects (missing argument)

Which Provides Better Support for Grammatical Correctness

The two mechanisms address different, complementary aspects of grammaticality rather than one being strictly superior. For enforcing agreement specifically, feature structures are the direct and necessary mechanism. For enforcing correct verb-argument structure, sub-categorization frames are essential. Full grammatical correctness in a production-grade grammar system requires both combined — as is done in unification-based grammar formalisms such as HPSG — since neither alone covers the other's class of errors.

Q5. Transition-Based vs. Graph-Based Dependency Parsing (Large-Scale)

Differences in Decision-Making and Performance

Transition-based parsing builds a dependency tree incrementally through a sequence of local shift/reduce-style decisions (e.g., using a stack and buffer), each decision made greedily based on the current parser state. This gives linear or near-linear time complexity, $O(n)$, making it very fast, but it is prone to error propagation since an early wrong decision cannot be undone. Graph-based parsing instead scores all possible head-dependent edges globally and extracts the highest-scoring spanning tree (e.g., using the Chu-Liu/Edmonds algorithm) over the whole sentence at once. This achieves higher accuracy through global optimization but at greater computational cost, typically $O(n^2)$ to $O(n^3)$.

Comparison Table

Aspect	Transition-Based Parsing	Graph-Based Parsing
Decision-making	Local, greedy, incremental	Global, scores all edges jointly
Speed	Fast, $O(n)$	Slower, $O(n^2)$ – $O(n^3)$
Accuracy	Prone to error propagation	Generally higher, globally optimal tree
Scalability to big data	Excellent	Limited by higher per-sentence cost

Which Is More Suitable for Large-Scale Applications

For large-scale NLP systems that must process massive datasets quickly, transition-based parsing is more suitable, since its linear-time complexity scales far better across huge volumes of text, especially when paired with modern neural scoring functions to mitigate its accuracy trade-off. Graph-based parsing remains preferable for smaller, accuracy-critical workloads where the higher computational cost per sentence is acceptable in exchange for globally optimal dependency trees.